

Distributed Systems

Lecture19: Leader Election

Mohammad Reza Shakournia

shakournia@ut.ac.ir

School of Electrical and Computer Engineering,
University of Tehran,
Tehran, Iran

Election Algorithms

- An algorithm requires that some process acts as a coordinator. The question is how to select this special process dynamically.
- In many systems, the coordinator is chosen manually (e.g., file servers). This leads to centralized solutions ⇒ single point of failure.
- **Teasers**
  - If a coordinator is chosen dynamically, to what extent can we speak about a centralized or distributed solution?
  - Is a fully distributed solution, i.e. one without a coordinator, always more robust than any centralized/coordinated solution?

Basic Assumptions

- All processes have unique id’s
- All processes know ID’s of all processes in the system (but not if they are up or down)
- Election means identifying the process with the highest ID that is up

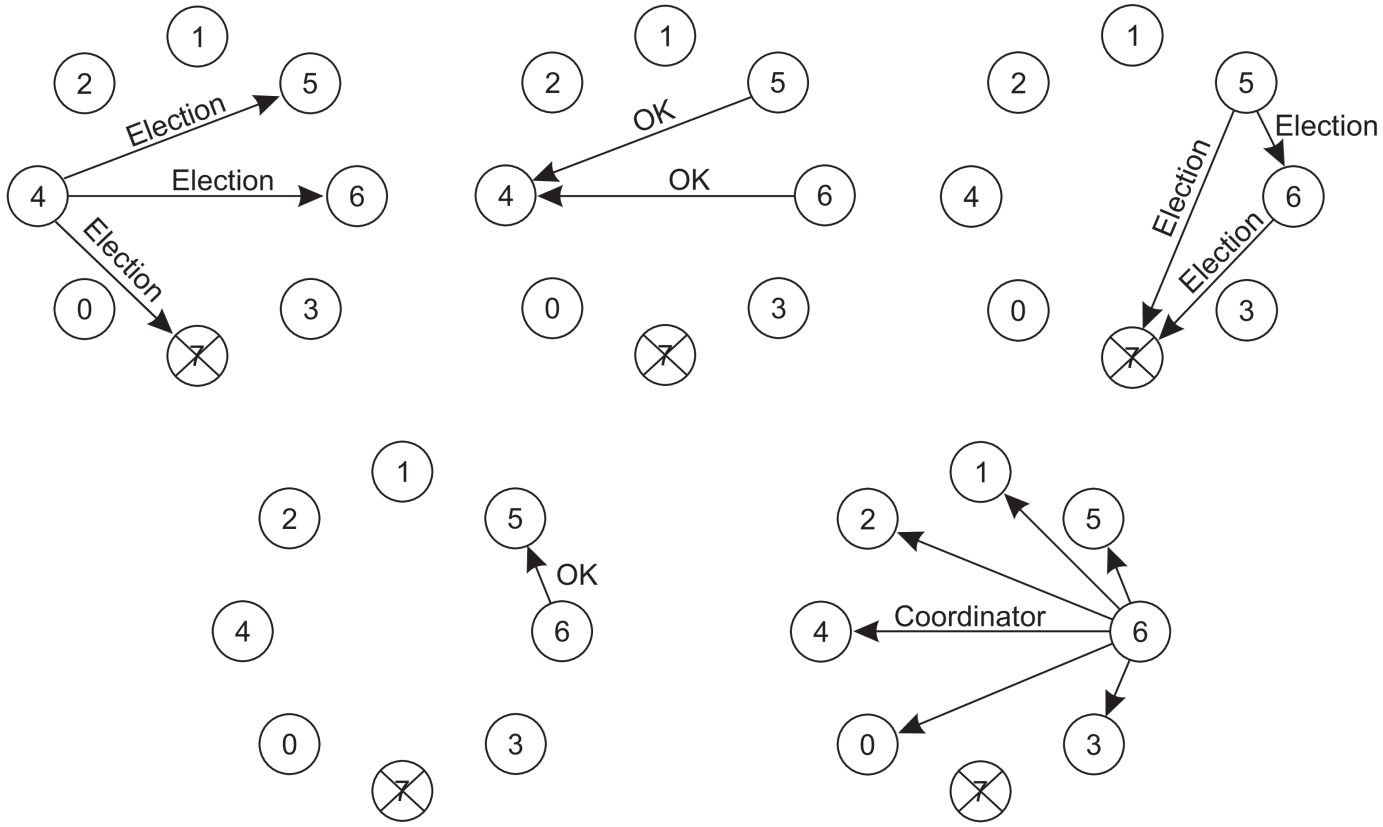
Election By Bullying

- Consider N processes {P0, . . . ,PN−1} and let id(Pk) = k. When a process Pk notices that the coordinator is no longer responding to requests, it initiates an election:

  1. Pk sends an ELECTION message to all processes with higher identifiers: Pk+1,Pk+2, . . . ,PN−1.
  2. If no one responds, Pk wins the election and becomes coordinator.
  3. If one of the higher-ups answers, it takes over and Pk ’s job is done.

Election By Bullying

- The bully election algorithm:

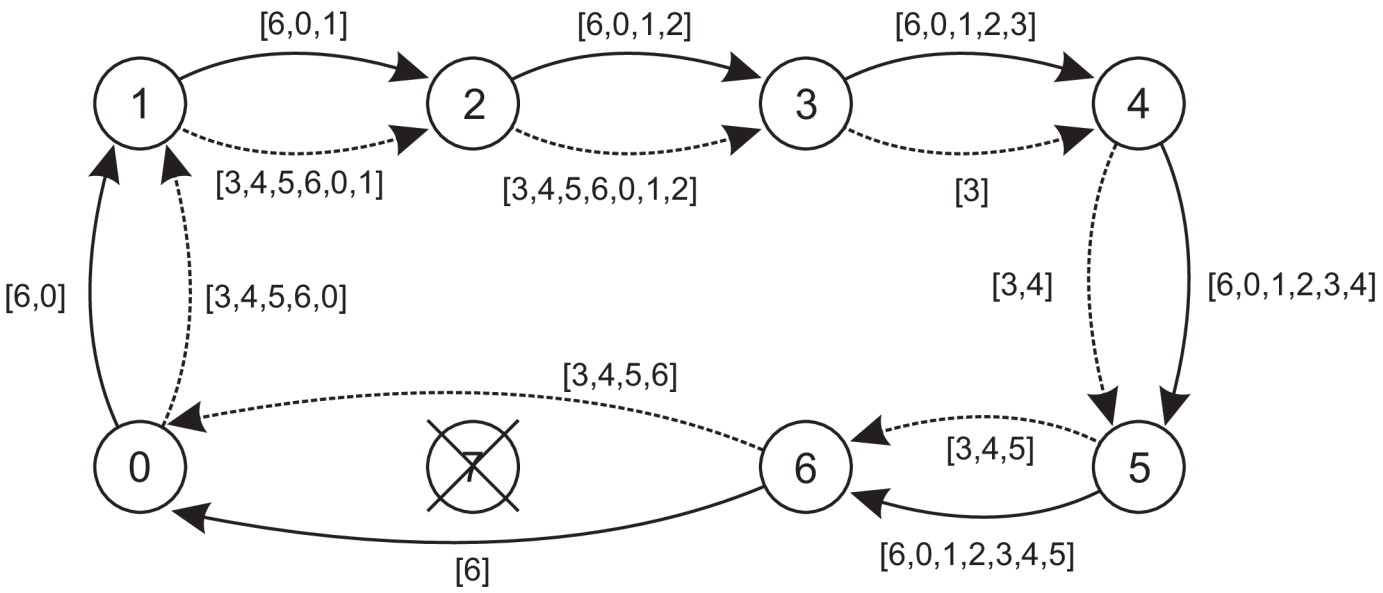


Election In a Ring

- Process priority is obtained by organizing processes into a (logical) ring. The process with the highest priority should be elected as coordinator.
  - Any process can start an election by sending an election message to its successor. If a successor is down, the message is passed on to the next successor.
  - If a message is passed on, the sender adds itself to the list. When it gets back to the initiator, everyone had a chance to make its presence known.
  - The initiator sends a coordinator message around the ring containing a list of all living processes. The one with the highest priority is elected as coordinator.

Election In a Ring

- Election algorithm using a ring



- The solid line shows the election messages initiated by P 6
- The dashed one, the messages by P3

Example: Leader Election in Zookeeper Server Group

- Each server s in the server group has an identifier id(s)
- Each server has a monotonically increasing counter tx(s) of the latest transaction it handled (i.e., series of operations on the namespace).
- When follower s suspects leader crashed, it broadcasts an ELECTION message, along with the pair (voteID,voteTX). Initially,
  - voteID ←id(s)
  - voteTX ←tx(s)
- Each server s maintains two variables:
  - leader(s): records the server that s believes may be final leader. Initially, leader(s)←id(s).
  - lastTX(s): what s knows to be the most recent transaction. Initially, lastTX(s)←tx(s).

Example: Leader Election in Zookeeper Server Group

- When s^* receives (voteID,voteTX)
- If lastTX(s^*) < voteTX, then s^* just received more up-to-date information on the most recent transaction, and sets
  - leader(s^*)←voteID
  - lastTX(s^*)←voteTX
- If lastTX(s^*) = voteTX and leader(s^*) < voteID, then s^* knows as much about the most recent transaction as what it was just sent, but its perspective on which server will be the next leader needs to be updated:
  - leader(s^*)←voteID
- When s^* believes it should be the leader, it broadcasts ⟨id(s^*), tx(s^*)⟩. Essentially, we're bullying.

Example: Leader Election In Raft

- We have a (relatively small) group of servers
- A server is in one of three states: follower , candidate, or leader
- The protocol works in terms, starting with term 0
- Each server starts in the follower state.
- A leader is to regularly broadcast messages (perhaps just a simple heartbeat)

Example: Leader Election In Raft

- **Selecting a new leader:**
- When follower s_* hasn't received anything from the alleged leader s for some time, s_* broadcasts that it volunteers to be the next leader, increasing the term by 1. s_* enters the candidate state. Then:
  - If leader s receives the message, it responds by acknowledging that it is still the leader. s_* returns to the follower state.
  - If another follower s_{**} gets the election message from s*, and it is the first election message during the current term, s** votes for s*. Otherwise, it simply ignores the election message from s_*. When s_* has collected a majority of votes, a new term starts with a new leader.
- By slightly differing the timeout values per follower for deciding when to start an election, we can avoid concurrent elections, and the election will rapidly converge.

Elections By Proof of Work

- **Basics:**
  - Consider a potentially large group of processes
  - Each process is required to solve a computational puzzle
  - When a process solves the puzzle, it broadcasts its victory to the group
  - We assume there is a conflict resolution procedure when more than one process claims victory.
- **Solving a computational puzzle:**
  - Make use of a secure hashing function H(m):
  - m is some data; H(m) returns a fixed-length bit string
  - computing h = H(m) is computationally efficient
  - finding a function H^{-1} such that m = H^{-1}(H(m)) is computationally extremely difficult
- Practice: finding H^{-1} boils down to an extensive trial-and-error procedure

Elections By Proof of Work

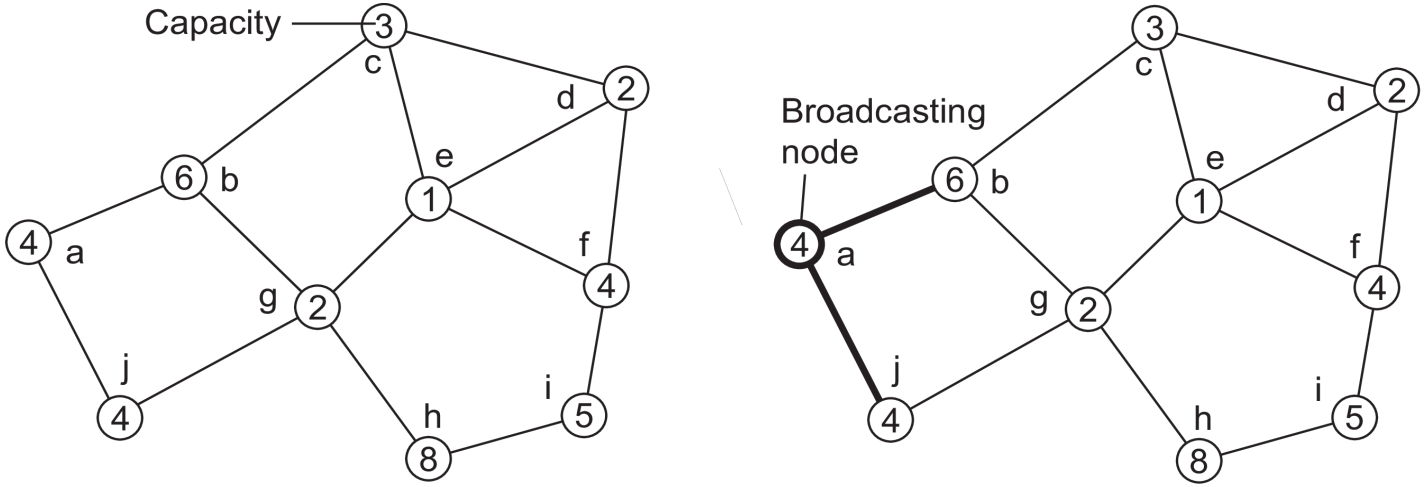
- **Controlled race:**
  - Assume a globally known secure hash function H^*. Let H_i be the hash function used by process P_i .
  - Task: given a bit string $h = H_i(m)$, find a bit string $\tilde{h}$ such that $h^* = H^*(H_i(\tilde{h} \odot h))$where:
    - h^* is a bit string with K leading zeroes
    - $\tilde{h} \odot h$ denotes some predetermined bitwise operation on $\tilde{h}$ and h.
- By controlling K, we control the difficulty of finding $\tilde{h}$. If p is the probability that a random guess for $\tilde{h}$ will suffice: $p = (1/2)^k$.
- **Current practice:**
- In many PoW-based blockchain systems, K = 64
  - With K = 64, it takes about 10 minutes on a supercomputer to find $\tilde{h}$
  - With K = 64, it takes about 100 years on a laptop to find $\tilde{h}$.

Elections By Proof of Stake

- **Basics:**
- We assume a blockchain system in which N secure tokens are used:
  - Each token has a unique owner
  - Each token has a uniquely associated index $1 \leq k \leq N$
  - A token cannot be modified or copied without this going unnoticed
- **Principle:**
  - Draw a random number $k \in \{1, \ldots, N\}$
  - Look up the process P that owns the token with index k. P is the next leader.
- The more tokens a process owns, the higher the probability it will be selected as leader.

A Solution for Wireless Networks

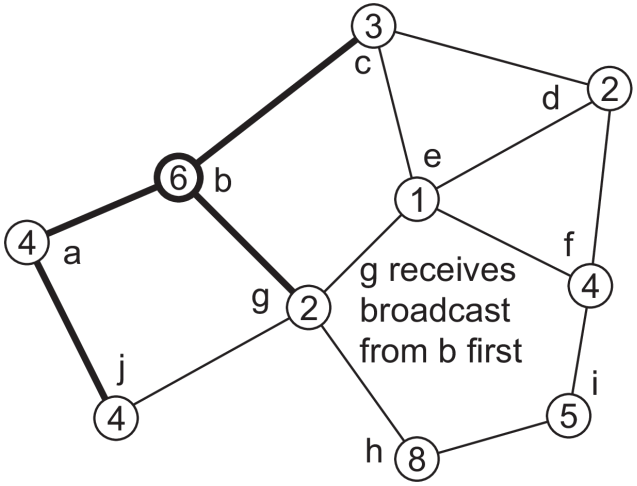
- A sample network

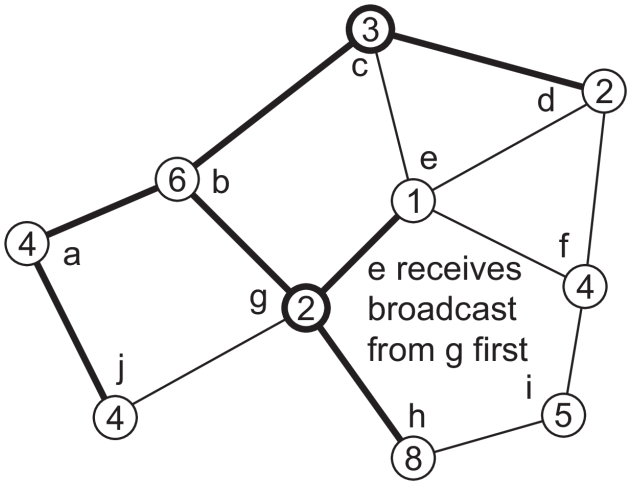


- Find the node with the highest capacity to select as the next leader.

A Solution for Wireless Networks

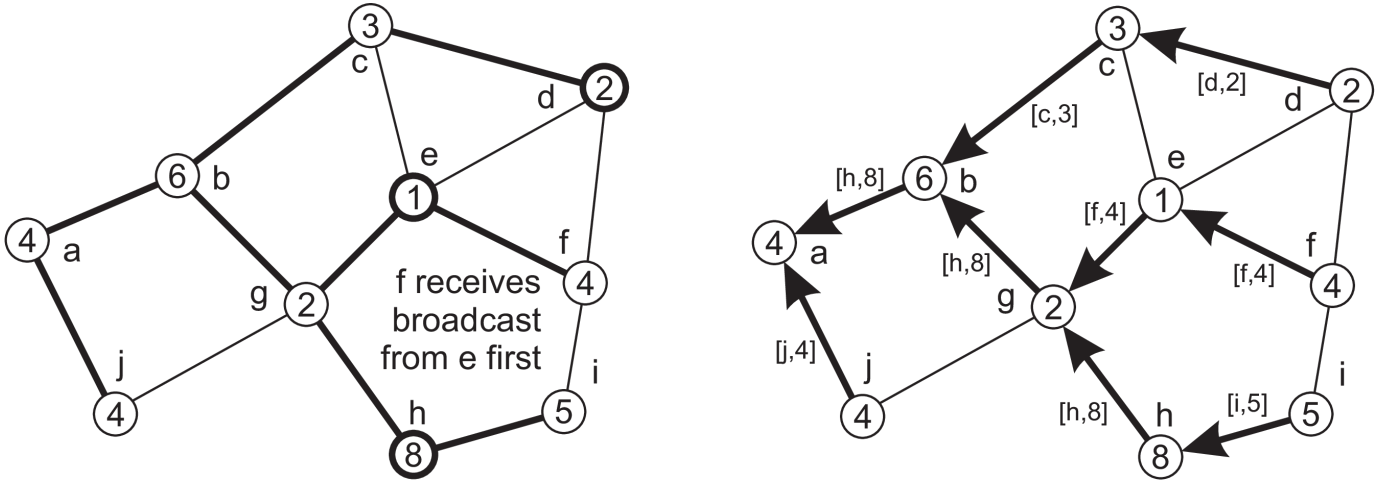
- A sample network





A Solution for Wireless Networks

- A sample network



- A node reports back only the node that it found to have the highest capacity.

Wrap-Up + What's Next

In the next lecture, we will discuss the Gossip-based coordination and Distributed event matching, including:

- Aggregation,
- A Peer-sampling Service,
- Gossip-based Overlay Construction,
- Secure Gossiping,
- Secure Publish-subscriber Solution.